

Go Security & Threat Mitigation Handbook

An Evergreen Engineering Specification & Code Reviewer Grading Standard

SECTION 1: DEPENDENCY VULNERABILITY MANAGEMENT

• **Call-Graph Scanning:** Enforce govulncheck in developer workflows and CI/CD pipelines. Unlike legacy manifest matchers that report unused transitive packages, govulncheck performs static call-graph analysis to trace active function calls from application entrypoints to vulnerable symbols, eliminating false positives.

```
// CI/CD Step: Run symbol-level call-graph vulnerability scanner
govulncheck ./...
```

• **Vulnerability DB (OSV Schema):** Ground vulnerability scans against the official Go Vulnerability Database (vuln.go.dev). The DB uses the Open Source Vulnerability (OSV) schema to provide curated, symbol-level vulnerability insights maintained directly by the Go Security Team.

• **Shift-Left CI/CD Gates:** Block pull requests automatically in CI by integrating govulncheck-action or parsing SARIF reports into code-scanning alerts. Do not rely solely on lockfile scanners that lack symbol reachability awareness.

```
# GitHub Actions step for symbol-level reachability scanning
- name: Run govulncheck
  uses: golang/govulncheck-action@v1
  with:
    go-version-input: '1.24'
    fail-on-vuln: true
```

SECTION 2: BOUNDARY VALIDATION & DOS PREVENTION

• **Payload Capping:** Wrap incoming HTTP request bodies with `http.MaxBytesReader` to enforce strict payload limits (e.g., 1 MB). This prevents memory exhaustion and unbounded read DoS attacks before buffering data.

```
func parseJSONBody(w http.ResponseWriter, r *http.Request, dst any) error {
    // Cap payload to 1MB to prevent memory exhaustion DoS
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20)
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(dst); err != nil {
        return fmt.Errorf("invalid request payload: %w", err)
    }
    if errors.Is(dec.Decode(&struct{}{}), io.EOF) == false {
        return errors.New("body must contain only a single JSON object")
    }
    return nil
}
```

• **Connection Timeouts (Slowloris Defense):** ALWAYS explicitly set `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout`, and `IdleTimeout` on `http.Server` instances. Relying on default 0-value timeouts leaves services vulnerable to connection-exhaustion attacks (Slowloris).

```
srv := &http.Server{
    Addr:           ":8080",
    Handler:        mux,
    ReadHeaderTimeout: 3 * time.Second, // Defends against Slowloris
    ReadTimeout:    10 * time.Second,
    WriteTimeout:    15 * time.Second,
    IdleTimeout:     120 * time.Second,
}
```

• **Safe JSON Decoding:** Call `decoder.DisallowUnknownFields()` to block mass-assignment vulnerabilities. Verify that a second decode call returns `io.EOF` to prevent trailing payload injection or concatenated JSON stream smuggling.

• **Declarative & Domain Validation:** Enforce struct validation tags via `go-playground/validator` for REST DTOs, use `protovalidate` with Common Expression Language (CEL) rules for gRPC Protobuf messages, and enforce unexported constructor functions for domain model invariants.

SECTION 3: INJECTION & PATH TRAVERSAL DEFENSE

• **SQL Injection Defense:** NEVER concatenate or format untrusted strings into SQL query strings (e.g., `fmt.Sprintf`). Enforce parameterized placeholder queries (`$1`, `?`) across database/sql execution methods.

```
// ■ BAD: SQL Injection Vulnerability
query := fmt.Sprintf("SELECT id FROM users WHERE email = '%s'", reqEmail)

// ■ GOOD: Parameterized query natively neutralizes injection
```

```
err := db.QueryRowContext(ctx, "SELECT id FROM users WHERE email = $1", reqEmail).Scan(&userID;)
```

- **Command & Path Traversal Protection:** Sanitize path inputs using `filepath.Clean`, enforce strict directory prefix allowlists via `strings.HasPrefix`, and NEVER pass untrusted user input directly to shell execution arguments or `exec.Command`.

```
func SafeFileRead(baseDir, userPath string) ([]byte, error) {
    clean := filepath.Clean(filepath.Join(baseDir, userPath))
    if !strings.HasPrefix(clean, filepath.Clean(baseDir)+string(filepath.Separator)) {
        return nil, errors.New("path traversal attempt detected")
    }
    return os.ReadFile(clean)
}
```

- **Contextual Auto-Escaping (XSS Defense):** Always use `html/template` for web rendering instead of `text/template`. `html/template` automatically contextually escapes HTML, JavaScript, CSS, and URI contexts to prevent Cross-Site Scripting (XSS).

SECTION 4: AUTHENTICATION, TOKENS & CSRF DEFENSE

- **Password Hashing & Constant-Time Comparison:** Hash passwords using memory-hard algorithms like Argon2id or bcrypt (cost ≥ 12). Always use `subtle.ConstantTimeCompare` when comparing secret tokens, HMAC signatures, or password hashes to eliminate timing side-channel attacks.

```
import "crypto/subtle"

// ■ Constant-time byte comparison eliminates timing attacks
func VerifyToken(provided, expected []byte) bool {
    return subtle.ConstantTimeCompare(provided, expected) == 1
}
```

- **JWT Validation Rules:** Explicitly validate JWT signatures, expiration (exp), issuer (iss), and audience (aud) claims. Reject the "none" algorithm explicitly and never store confidential PII in unencrypted JWT claims.
- **CSRF & WebSocket Security (CSWSH):** Enforce strict `SameSite=Strict` or `SameSite=Lax` cookie attributes. For WebSockets, validate the HTTP Origin header inside `websocket.Upgrader.CheckOrigin` callbacks to prevent Cross-Site WebSocket Hijacking.

```
var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool {
        origin := r.Header.Get("Origin")
        return origin == "https://app.example.com" // Validate origin strictly
    },
}
```

- **Enumeration Prevention:** Return generic error responses (e.g., "invalid credentials") on authentication failures. Never distinguish between 'user not found' and 'incorrect password' to prevent user enumeration.

SECTION 5: SECRETS & TELEMETRY SECURITY

- **Zero Committed Credentials:** Prohibit hardcoded API keys, database passwords, or private keys in source code or committed repository .env files. Enforce automated pre-commit secret scanners (e.g., gosec, git-leaks).
- **Runtime Secret Injection:** Retrieve production secrets dynamically at application runtime using Secret Manager SDKs (1Password SDK, AWS Secrets Manager, Vault) into RAM, or inject them via ephemeral tmpfs RAM mounts or CLI wrappers (Doppler, 1Password CLI).

```
// Fetch secret directly to memory using 1Password SDK
client, err := onePassword.NewClient(ctx, opts...)
secret, err := client.Secrets.Resolve(ctx, "op://vault/db/password")
```

- **PII Redaction in Telemetry:** Implement the `slog.LogValuer` interface on sensitive domain models and credentials structs so that structured loggers automatically mask PII, tokens, and passwords during serialization.

```
type UserCredentials struct {
    Username string
    Password string
}

// LogValuer masks sensitive fields during slog serialization
func (c UserCredentials) LogValue() slog.Value {
    return slog.GroupValue(
        slog.String("username", c.Username),
        slog.String("password", "[REDACTED]"),
    )
}
```

SECTION 6: CONTAINERIZATION & SUPPLY CHAIN

- **Minimal Multi-Stage Builds:** Compile static Go binaries (CGO_ENABLED=0) and copy only the compiled executable into empty scratch or gcr.io/distroless/static base images. Eliminating shell binaries, package managers, and debugging tools dramatically reduces attack surface.
- **Reproducible Builds & Path Stripping:** Pass the -trimpath flag to go build. This strips build-host absolute file system paths from the binary, preventing internal directory path leakage in stack traces and enabling byte-for-byte reproducible builds.

```
# Multi-stage Dockerfile compiling minimal static binary
FROM golang:1.24-alpine AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" -o /service ./cmd/server

FROM gcr.io/distroless/static-debian12
USER 10001:10001
COPY --from=builder /service /service
ENTRYPOINT ["/service"]
```

- **Non-Root User Enforcement:** Explicitly configure containers to execute as an unprivileged non-root user (e.g., USER 10001:10001). Never run Go microservices as root in production containers.
- **Static Security Analysis (gosec):** Integrate gosec into golangci-lint configurations to automatically audit Go ASTs for hardcoded credentials (G101), weak TLS settings (G402), unsafe file permissions (G302), and poor random number generation (G404).

SECTION 7: SECURITY REVIEWER GRADING MATRIX

Category	Review Standard	Code Smell / Anti-Pattern	Idiomatic Remediation
Vulnerabilities	Call-graph reachability scanning via govulncheck	Relying only on lockfile manifest scanners; ignoring active call paths	govulncheck ./... in CI/CD pipelines
Payload Capping	Enforce max request body size limits	Unbounded io.ReadAll(r.Body) allowing memory DoS	r.Body = http.MaxBytesReader(w, r.Body, 1<<20)
Server Timeouts	Set explicit ReadHeaderTimeout on http.Server	Default zero-value http.Server timeouts (Slowloris vulnerability)	ReadHeaderTimeout: 3 * time.Second
JSON Safety	Disallow unknown fields; verify single object	Silently ignoring unknown fields or accepting concatenated streams	dec.DisallowUnknownFields() + io.EOF check
SQL Safety	Enforce parameterized placeholders (\$1, ?)	String formatting SQL queries (fmt.Sprintf)	db.QueryRowContext(ctx, "WHERE id = \$1", id)
Path Safety	Clean paths and verify base directory prefix	Passing user input directly to os.ReadFile or exec.Command	filepath.Clean + strings.HasPrefix check
Secrets / Timing	Constant-time byte comparison for secrets	Standard == string comparison on HMACs or tokens	subtle.ConstantTimeCompare(a, b) == 1
CSRF & WS	Origin checks and SameSite cookie policies	Allowing all origins in WebSocket CheckOrigin	Strict origin validation in CheckOrigin
Telemetry PII	Implement slog.LogValuer to mask sensitive data	Logging raw user structs containing passwords or tokens	func (u User) LogValue() slog.Value
Supply Chain	Minimal scratch images, trimpath, non-root user	Heavy base images with shell tools running as root user	CGO_ENABLED=0 + -trimpath + USER 10001